Practice · Whole programs 6

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What the word program does

Read the whole program before you answer.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int count_vowels(char s[]) {
    int count = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = tolower(s[i]);
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
            count++;
        }
    }
    return count;
}

int main(void) {
    char word[30] = "Programming";
    char copy[30];
    strcpy(copy, word);
    strcat(copy, "!");
    printf("%s has %d letters\n", word, (int) strlen(word));
    printf("vowels: %d\n", count_vowels(word));
    printf("%s\n", copy);
    printf("same: %d\n", strcmp(word, copy) == 0);
    return 0;
}
```

Why does `count_vowels` call `tolower` first?

- To change the letters of `word` to small letters for good.
- **Right answer:** So that a capital vowel like `A` is counted too, with only five small letters in the test.
- Because `==` cannot compare two `char` values.
- Because `s[i]` is a string and `tolower` makes it a `char`.

Feedback: `tolower` gives the small letter of a character, so `'A'` and `'a'` both become `'a'` and one test with five small vowels is enough. It changes only the copy in `c`, not the string. Class 11, slides 28 and 61-62.

(mc) L2 · The word program prints

Same program as before. What are the four lines it prints?

- `Programming has 12 letters`, `vowels: 3`, `Programming!`, `same: 1`
- `Programming has 11 letters`, `vowels: 4`, `Programming!`, `same: 0`
- **Right answer:** `Programming has 11 letters`, `vowels: 3`, `Programming!`, `same: 0`

- `Programming has 30 letters, vowels: 3`, `Programming, same: 1`

Feedback: `strlen` counts the letters before the end mark: 11 (not 30, the size of the array). The vowels are o, a and i: 3. `copy` got `"!"` added, so `word` and `copy` differ, `strcmp` is not 0, and the comparison with 0 gives 0. Class 12, slides 9, 14, 16 and 27.

(mc) L2 · Find the bug: copy = word

Someone writes `copy = word;` instead of `strcpy(copy, word);`. What happens?

- It works the same way.
- It compiles, but `copy` stays empty.
- **Right answer:** gcc stops with an error: you cannot give an array a new value with `=`.
- gcc warns, and the program prints `Programming!` anyway.

Feedback: An array cannot be given a new value with `=`; gcc stops with `assignment to expression with array type`. The letters are copied with `strcpy(to, from)`. Class 12, slides 12 to 14.

(mc) L2 · All capitals

Same program as before. What does `count_vowels("AEIOU")` return?

- `0`, because the test has only small letters
- **Right answer:** `5`
- `1`
- It does not compile: a string in quotes cannot be passed to a function.

Feedback: `tolower` turns each capital into a small letter before the test, so all five are counted. A string in quotes can go straight to a function that takes a string. Run with the course's gcc: 5. Class 11, slides 61-62; class 12, slide 36.

(fib) L3 · Complete the copy

Same program as before. Complete the line that copies `word` into `copy`. Type only the name of the function.

```
[strcpy](copy, word);
```

Feedback: `strcpy(to, from)` copies the letters and the end mark, in the same order as `to = from`. Then `strcat` adds `"!"` to the end of `copy`. Class 12, slide 14.

(fib) L3 · Complete the length

Same program as before. Complete the call that gives the number of letters in `word`. Type only the name of the function.

```
printf("%s has %d letters\n", word, (int) [strlen](word));
```

Feedback: `strlen` counts the characters before the end mark: 11 for `"Programming"`. `sizeof(word)` would give 30, the boxes of the whole array. Class 12, slides 9 and 10.

(mc) L1 · What the sales program does

Read the whole program before you answer.

```
#include <stdio.h>
```

```
#define ROWS 3
#define COLS 4

int main(void) {
    int sales[ROWS][COLS] = {{2, 5, 1, 4},
                             {3, 0, 6, 2},
                             {7, 1, 2, 5}};
    for (int c = 0; c < COLS; c++) {
        int sum = 0;
        for (int r = 0; r < ROWS; r++) {
            sum += sales[r][c];
        }
        printf("day %d: %d\n", c + 1, sum);
    }
    return 0;
}
```

Three shops (rows) sold things on four days (columns). What does the program print?

- The total of each shop.
- **Right answer:** The total of each day: one line per column.
- The total of the whole table.
- The best day of each shop.

Feedback: The column loop is outside, so there is one answer per column: one total per day. The inner loop goes down the three shops. Class 13; class 14, slides 28 to 30.

(mc) L2 · The sales program prints

Same program as before. What are its four lines?

- **Right answer:** `day 1: 12`, `day 2: 6`, `day 3: 9`, `day 4: 11`
- `day 1: 12`, `day 2: 11`, `day 3: 13`, `day 4: 0`
- `day 0: 12`, `day 1: 6`, `day 2: 9`, `day 3: 11`
- `day 1: 2`, `day 2: 5`, `day 3: 1`, `day 4: 4`

Feedback: Column 0 holds 2, 3 and 7: 12. Column 1: 5 + 0 + 1 = 6. Column 2: 1 + 6 + 2 = 9. Column 3: 4 + 2 + 5 = 11. The program prints `c + 1`, so the days are numbered from 1. 12, 11 and 13 are the shop totals (rows), a different question. Class 13.

(mc) L2 · Find the bug: the sums grow

Someone moves `int sum = 0;` above the outer loop. What does the program print now?

- The same four lines.
- `day 1: 12` four times.
- **Right answer:** `day 1: 12`, `day 2: 18`, `day 3: 27`, `day 4: 38`
- Nothing: it does not compile.

Feedback: With `sum = 0` only once, the sum is never reset, so each day adds to the total of the days before: 12, then 12 + 6 = 18, then 27, then 38. A new answer per column needs a new `sum = 0` inside the outer loop. Class 13; class 14, slide 15.

(fib) L3 · Complete the outer loop

Same program as before. Complete the outer loop so it visits every column. Type only the name that goes after `c <`.

```
for (int c = 0; c < [COLS]; c++) {
```

Feedback: The columns are numbered 0 to `COLS - 1`, so the loop runs while `c < COLS`. Writing the name, not 4, keeps the program right if the number of days changes. Class 13.

(mc) L1 · What the heat map program does

Read the whole program before you answer.

```
#include <stdio.h>

#define ROWS 3
#define COLS 4

int is_inside(int r, int c) {
    return r >= 0 && r < ROWS && c >= 0 && c < COLS;
}

int hot_around(int t[][COLS], int r, int c, int limit) {
    int count = 0;
    for (int dr = -1; dr <= 1; dr++) {
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) {
                continue;
            }
            if (is_inside(r + dr, c + dc) && t[r + dr][c + dc] > limit) {
                count++;
            }
        }
    }
    return count;
}

int main(void) {
    int temp[ROWS][COLS] = {{20, 31, 25, 18},
                            {33, 22, 35, 30},
                            {19, 28, 21, 34}};
    printf("%d\n", hot_around(temp, 0, 0, 30));
    printf("%d\n", hot_around(temp, 1, 1, 30));
    printf("%d\n", hot_around(temp, 2, 3, 30));
    return 0;
}
```

What does `hot_around(temp, r, c, limit)` return?

- The highest temperature around the cell.
- **Right answer:** How many neighbour cells of `(r, c)` are hotter than `limit`.
- How many cells in the whole map are hotter than `limit`.

- 1 if the cell itself is hotter than `limit`, else 0.

Feedback: The two loops visit the 8 positions around `(r, c)`, skip the cell itself with `continue`, and count a neighbour when it is on the map and its temperature is greater than `limit`. Class 14, slides 39 and 42.

(mc) L2 · The heat map program prints

Same program as before. What are its three lines?

- `2`, `3`, `2`
- `3`, `3`, `1`
- **Right answer:** `2`, `3`, `1`
- `2`, `4`, `1`

Feedback: Around (0, 0): 31 and 33 are above 30, 22 is not: 2. Around (1, 1): 31, 33 and 35: 3. Around (2, 3): 35 is above 30, but 30 is not (the test is greater than, not greater or equal), and 21 is not: 1. Class 14, slides 39 to 44.

(mc) L2 · Find the bug: the order

Someone writes the condition the other way round: the temperature test first, then `is_inside`. What is wrong with it?

- Nothing: `&&` gives the same answer in any order.
- **Right answer:** For a position off the map, the temperature is read before the check, from outside the array.
- It does not compile.
- It counts the cell itself.

Feedback: `&&` stops at the first false part. With `is_inside` first, a position off the map stops the test before the map is read. In the other order the program reads outside the array first, and C does not stop it. Check first, then read. Class 14, slides 36, 37 and 45.

(mc) L2 · Without the continue

Someone deletes the `if` with `continue`. What does `hot_around(temp, 1, 2, 30)` count now? The cell in row 1, column 2 holds 35.

- The same as before.
- **Right answer:** One more than before, because the cell itself (35) is now counted too.
- Zero, because the loop stops.
- It does not compile.

Feedback: Without the `continue`, the position with `dr` 0 and `dc` 0 is tested too. That is the cell itself, it is on the map and 35 is above 30, so it is counted as if it were its own neighbour. Class 14, slides 33 and 39.

(fib) L3 · Complete the corner answer

Same program as before, with the original code. Type what `hot_around(temp, 0, 3, 30)` returns. The corner (0, 3) holds 18; its neighbours hold 25, 35 and 30.

Count: [1]

Feedback: A corner has 3 neighbours. Only 35 is greater than 30; 30 itself is not, because the test is `>`. So 1. Class 14, slides 35 and 39.

(fib) L3 · The last row inside

Same program as before. `ROWS` is 3. What is the largest row number for which `is_inside` can return 1? Type the number.

Last row: [2]

Feedback: `is_inside` tests `r < ROWS`, so with `ROWS` 3 the rows inside are 0, 1 and 2. Rows start at 0, so the last one is `ROWS - 1`. With `<=` instead, row 3 would count as inside and the program would read one row past the end. Class 14, slide 38.